

INF2204 — Computer Systems Experiments, Methodologies, Performance Measurements and Evaluation

Assignment 1 — Experiment Project Proposal

1. Group description

Group members: Anna Reedt — *group formation still in progress; see note to course staff regarding late group placement.* Additional members and their names will be added here once the group is finalized.

2. Introduction

2.1 Motivation and goal

Real-time cycling training platforms (e.g. Zwift, TrainerRoad, Wahoo's ecosystem) rely on continuous streams of small telemetry packets — power, cadence, heart rate — sent from ANT+/Bluetooth Low Energy sensors through a local device and, in many setups, over a network before reaching the software that renders feedback to the athlete. Because pacing decisions, structured-workout targets, and even competitive outcomes in virtual racing depend on this data arriving promptly and completely, the transport layer's behavior has a direct, practical effect on the athlete's experience.

Despite this, the transport-layer trade-offs involved are rarely visible to athletes or to hobbyist developers building their own training tools. TCP guarantees delivery and ordering but can introduce retransmission delay and head-of-line blocking under loss; UDP avoids that delay but offers no such guarantees, risking dropped or out-of-order readings. A data stream that looks smooth but is stale, or one that is timely but incomplete, can each mislead an athlete's decisions in different ways. This project aims to build a small, controlled telemetry pipeline that mimics a power/cadence sensor feeding a training display application, and to measure how transport protocol choice, network conditions, and the addition of transport-layer encryption affect the timeliness and completeness of data delivery.

2.2 Research question(s)

- **RQ1:** Under identical simulated network conditions (bandwidth, latency, jitter, packet loss), how do TCP and UDP compare in end-to-end latency and data completeness for a stream of small, fixed-rate telemetry packets representative of cycling sensor data (e.g. 1–4 Hz, ≤ 20 bytes per reading)?
- **RQ2:** As packet loss and jitter increase, how does each protocol's ability to deliver telemetry within a “usable” latency budget for real-time feedback (e.g. under 250 ms) degrade?
- **RQ3:** What latency and throughput overhead does adding transport-layer encryption (TLS over TCP, DTLS over UDP) introduce to this pipeline, and is that overhead significant relative to the latency budget identified in RQ2?

3. Related work

Two bodies of prior work inform this study directly, and course material provides a template for the kind of small, well-scoped system we aim to build.

On transport protocol choice (RQ1–RQ2): Wirges and Dettmar compare TCP and UDP behaviour over Narrowband IoT (NB-IoT) networks [1], a setting characterised by high latency and constrained bandwidth similar in spirit to the lossy conditions we simulate with `netem`, motivating transport-layer choice as a first-order design decision for constrained, real-time-adjacent links. More directly relevant is sFlow's analysis of UDP versus TCP for real-time streaming telemetry [2], which reports that at 14% simulated packet loss, average TCP message delay exceeds 1 second while UDP holds steady at roughly 160 microseconds, and that TCP throughput collapses from 96 to 10 messages/second at 20% loss while UDP degrades only to 77 messages/second under the same conditions. This is strong practical evidence for the head-of-line-blocking mechanism we hypothesise will dominate RQ1–RQ2's results — though it is an industry analysis rather than a peer-reviewed study, and does not isolate jitter separately from loss, a gap our controlled experiment is designed to close.

On transport-layer encryption overhead (RQ3): Restuccia, Tschofenig, and Baccelli evaluate DTLS and TLS 1.3 against their 1.2 predecessors on low-power IoT microcontrollers [3], measuring data transmitted over the air, memory footprint, and energy consumption, and conclude that where 1.3 does add overhead relative to 1.2, that overhead is “quite reasonable,” with some configurations even reducing resource use. Their study,

however, targets embedded microcontrollers rather than general-purpose hosts, and measures resource consumption rather than end-to-end latency under adverse network conditions — the gap RQ3 targets directly.

Finally, our approach to modelling and scoping the system follows the method introduced in this course through Guégan and Raïs's simulation of a constrained Arctic sensor node [4], read for Colloquium 2. That paper studies a battery-powered node's trade-off between local storage and immediate transmission under unreliable connectivity. Our project asks a related but distinct question: rather than storage-vs-transmission timing, we isolate transport protocol and encryption choice as the independent variables, using a synthetic cycling-telemetry workload instead of an environmental sensor, so as to keep our system and research questions clearly original relative to the assigned reading.

4. System

The system under study is a synthetic cycling-telemetry pipeline: a chain of components that mimics a power/cadence sensor feeding a real-time training display, built so that transport-layer behaviour can be measured under controlled, repeatable network conditions rather than on a real, uncontrollable Bluetooth/ANT+ link. As emphasised in the course material on scoping, the boundary of a system follows from the question being asked of it, not from the system itself; since RQ1–RQ3 all concern the data path from sensor to display, the boundary is drawn around the software pipeline that produces, transports, and receives telemetry, while the physical sensor and the visual training app are represented only schematically.

Components (inside the boundary). A **Sensor Emulator** generates synthetic power/cadence readings at a fixed rate (1–4 Hz). A **Sender** serialises each reading and transmits it using the transport under test (TCP or UDP, optionally wrapped in TLS or DTLS). A **Network Emulator**, implemented with Linux `tc/netem`, applies controlled latency, jitter, and packet loss between sender and receiver. A **Receiver** accepts and deserialises incoming packets, and a **Logger** timestamps every reading on arrival, producing the raw data from which latency, completeness, and out-of-order arrivals are computed.

Outside the boundary, but relevant. The host operating system's scheduler and kernel network stack influence measured timing (e.g. thread scheduling jitter, NIC buffering) but are not part of the designed system and cannot be fully controlled; this is treated as measurement noise rather than modelled directly.

Outside the boundary, and irrelevant. Hardware cost, the display/GPU used to view results, unrelated background processes, and the machine's physical location have no bearing on the transport-layer measurements and are excluded from the study.

Inputs and outputs. The system's input is a synthetic reading (power, cadence, timestamp) generated internally — no external input crosses the boundary, since the sensor itself is simulated for reproducibility. The output is the Logger's timestamped receive record, from which end-to-end latency, delivery completeness, and jitter are computed.

Actors. The **Researcher** configures experiments and analyses logs. The **Athlete** does not interact with the system directly but motivates its design, as a proxy for the real sensor-to-display path.

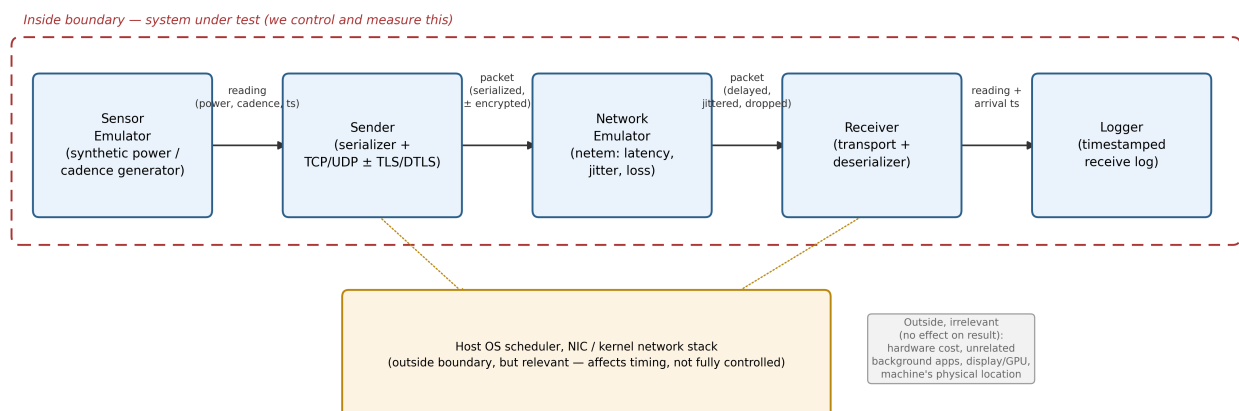


Figure 1. Block diagram of the cycling-telemetry pipeline under study. The dashed line marks the system boundary for RQ1–RQ3; the dotted lines indicate an outside-but-relevant influence on timing.

5. Early environment and scope

Since the system under study is fully synthetic (Section 4), the experimental setup does not depend on specialised or hard-to-access hardware. The following is an early sketch and will likely be refined once the group and its available machines are confirmed.

- **Hardware:** a single standard laptop suffices for development. For a real network hop rather than loopback traffic, sender and receiver will run in separate Linux network namespaces connected by a virtual Ethernet (veth) pair, with `tc/netem` applied to that link. As a stretch goal, key experiments may be repeated across two physical machines on the same LAN.
- **Software:** Python 3 (`socket`, `ssl`, `asyncio`) and/or C (BSD sockets, OpenSSL) for sender/receiver; Linux `tc/netem` for network emulation (group members on Windows/macOS will need a Linux VM or WSL2); Python (`pandas`, `matplotlib`) for log analysis; Git/GitHub for version control and hand-in.
- **Datasets:** none required — telemetry is synthetic, generated at a fixed rate in line with typical ANT+/BLE cycling power meter broadcast rates, so all conditions can be repeated exactly.
- **Access needs:** none beyond standard development tools; the LAN-based stretch goal only requires two machines on the same local network.
- **Controlled context:** independent variables are transport protocol (TCP, UDP, TCP+TLS, UDP+DTLS) and network condition profile (baseline, added latency, jitter, loss, combined); controlled variables are packet rate and payload size; dependent variables are end-to-end latency, delivery completeness, and out-of-order count. Each configuration will be repeated across multiple runs to account for host-OS scheduling noise, and results will be reported with variance rather than single-run values.

6. Early timeline

This plan is anchored to the course's official schedule (handouts and deadlines for Assignments 1–3, the two class presentation checkpoints, and the oral exam week), with our own internal milestones filled in between.

- **W34–W37 (17 Aug – 7 Sep):** course introduction, literature review, systems analysis, and experimental design (lectures/colloquia); Assignment 1 (this proposal) handed out Mon 24 Aug, due Mon 7 Sep.
- **W37–W40 (8 Sep – 28 Sep, Assignment 2):** Assignment 2 handed out Tue 8 Sep, due Mon 28 Sep; a “Students project presentation” checkpoint also falls on Mon 28 Sep. Internally: finalise group and topic with course staff, refine RQ1–RQ3 and the system design (Section 4) with TA feedback, and implement a first working Sensor Emulator→Sender→Receiver→Logger path over loopback (no induced network conditions yet), ready to present.
- **W40–W47 (29 Sep – 16 Nov, Assignment 3):** Assignment 3 handed out Wed 29 Sep, due Mon 16 Nov — the main implementation and experimentation phase. Internally: build the Linux network-namespace + `netem` environment, add TCP/UDP and TLS/DTLS variants, run the full experiment matrix (protocol × network condition) with repeated trials, and analyse latency, completeness, and jitter.
- **W47–W48 (16 Nov – 23 Nov):** consolidate results into figures/tables and prepare for the “Student Conference: results presentation” on Mon 23 Nov.
- **W49–W51 (24 Nov – 14 Dec):** finalise the written report and the GitHub repository (commit history, documentation, reproducibility instructions); prepare for the oral exam (week of 14 Dec).

7. Discussion

- **Group formation:** at the time of writing, the group is not yet finalised due to a scheduling conflict; this creates schedule pressure and the final group size/composition may still shift, which could affect division of labour and pacing on later assignments.
- **Topic originality (FIFO):** since group topics are validated on a first-come basis, there is a risk another group has chosen an overlapping topic; we plan to confirm originality directly with course staff.
- **Environment portability:** `netem`-based emulation requires Linux; group members on Windows/macOS will need a VM or WSL2, which could introduce inconsistencies between members' environments and complicate reproducibility.
- **External validity:** emulated network conditions and synthetic sensor data simplify real wireless links (e.g. real ANT+/BLE radio interference); results may not directly generalise to a physical sensor deployment, though this is a deliberate trade-off for reproducibility.

- **Confounding variables:** host-OS scheduling noise (Section 4) could add measurement variance that is hard to fully separate from injected network conditions; repeated trials mitigate but do not eliminate this.
- **Implementation skill/time risk:** as developers still building up Python/C experience, the group may need extra time for the C-based components and OpenSSL/TLS integration, which could affect the timeline above.
- **Implementation-specific results:** TLS/DTLS library behaviour (e.g. handshake cost) can vary by implementation and version, so encryption-overhead results may be somewhat implementation-specific rather than universal.

References

- [1] J. Wirges and K. Dettmar, "Performance of TCP and UDP over Narrowband Internet of Things (NB-IoT)," IEEE, 2019/2020. *(verify exact venue/year before final submission)*
- [2] sFlow, "UDP vs TCP for real-time streaming telemetry," sflow.com blog, 2022.
- [3] G. Restuccia, H. Tschofenig, and E. Baccelli, "Low-Power IoT Communication Security: On the Performance of DTLS and TLS 1.3," 9th IFIP/IEEE PEMWN, Dec. 2020.
- [4] Guégan and Raïs, "Simulation of Distributed Systems in Constrained Environments Using ESDS: the Arctic Tundra Case," iThings 2023.